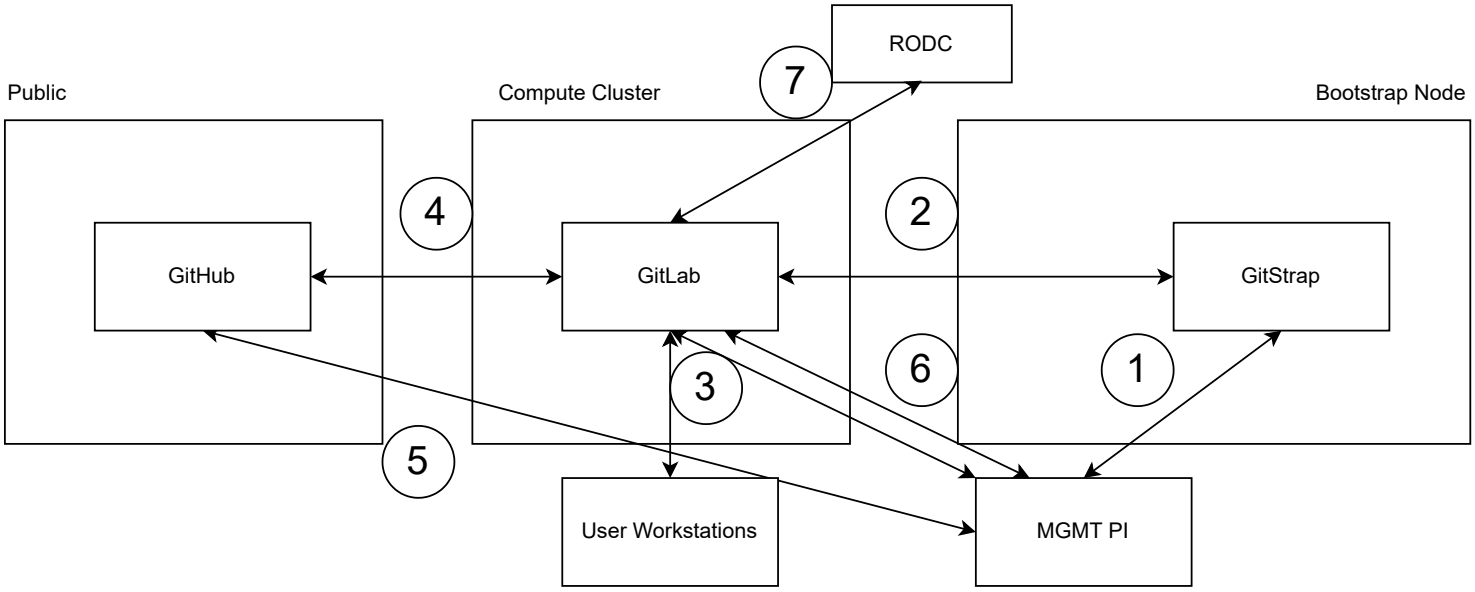
Code Ecosystem			
Subsystem Code	co	Component Codes	git = Git glab = GitLab



Data Flows			
ID	Port/Proto	Description	Controls
1	22/TCP	Emergency fall-back Git-Over-SSH to the lightweight bootstrap git service	Local firewall rules restrict access to only MGMT PI and GitLab nodes using key-based verification. Includes pre-receive hook to block vault files.
2	22/TCP	Git-to-GitLab sync: GitLab pushes to GitStrap automatically; GitStrap pushes to GitLab for recovery/restore with human intervention	SSH keypairs (deploy key) scoped to IaC repo.
3	443/TCP	Users push to the local GitLab instances	Individual, domain-backed user accounts.
4	443/TCP	GitLab CI filters repository and pushes public-safe artifacts to GitHub	PAT tied to org GitHub account stored as a masked CI variable. Leverages white-list approach for defining public-safe artifacts
5	443/TCP	Archived method to push public-safe artifacts direct to public from MGMT PI; inactive	PAT tied to org GitHub account. Leverages white-list approach for defining public-safe artifacts
6	443/TCP	MGMT PI queries for pending deployments, claims jobs, and reports results after execution	TLS and token-based API auth to ensure MGMT PI is the only entity claiming and running deployment jobs. Will leverage different credentials to list jobs and download resources (read-only) and claim and report results (read-write).
7	389/TCP 646/TCP	GitLab connection to Identity Subsystem for centralized authentication	LDAP(S) with a dedicated bind account that only supports GitLab.

Infrastructure CI Execution Flow			
ID	Actor	Action	Result
1	Developer User	Submits Merge Request to infrastructure repo	MR created and ready for review
2	CI Pipeline	Reviews MR, conducts repo-specific tests, and ensures repo is ready for merge	CI reports pipeline pass
3	Reviewer User	Reviews MR and pipeline results	Accepts merge
4	CI Pipeline	Reviews new commit vs previous state to compute playbook impact	CI emits one or more deployment requests[1,2,3] based on computed playbook impact
5	MGMT PI	Scans for deployment requests at set interval	Observes new deployment request[1], claims job[2], and downloads artifacts
6	MGMT PI	Runs ansible playbooks based on CI job specifications	State enforced per new IaC baseline
7	MGMT PI	Collects jobs results and pushes to GitLab	CI Pipeline records and displays job results

[1] Deployment requests will reference a tag or commit SHA, not branch head, to ensure deterministic execution if branch changes between job issuance and job execution.

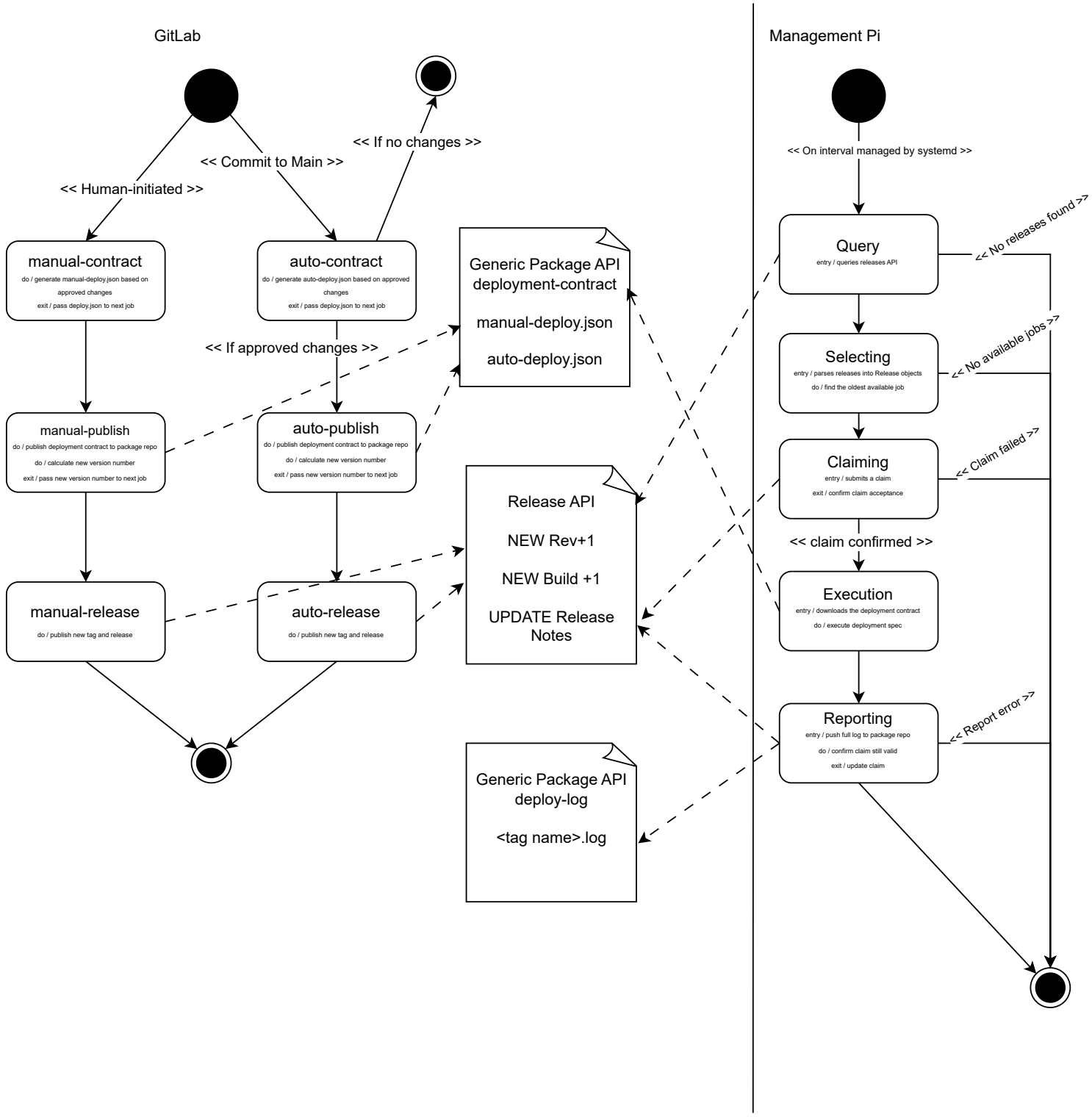
[2] Deployment requests will be configured such that they are claimed once and executed once. Re-runs will require human intervention.

[3] Initial implementation will likely follow GitLab issues with labels; exact specification to follow in a future revision

Pipeline Overview

The CI Pipeline for the Infrastructure Repo enables changes to the infrastructure baseline from lesser privileged environments and without any contact with the authentication layer. The pipeline triggers on commits to the main branch or via manual triggers. On commit to main, the pipeline will automatically identify files that were changed and attempt to map those files to changes that can be executed automatically: for example, adding/remove static dns records, changing the port configuration on a switch, or managing VM specifications. When triggered manually, users can specify any combination of playbook and tag to execute. The result increments the Build or Revision version and creates a new release, which includes the details necessary to implement the change.

The Deployment Executor, hosted on the Management Pi, queries GitLab for releases with changes to implement, claims them, and executes them, delivering the details back to the Release for state tracking and to support troubleshooting.



Deployment Contract

The Deployment Contract is an auditable JSON object that provides a reference to the code base and a specific set of actions to implement via ansible. Contracts are automatically generated based on files changed in a commit or merge to main, or based on user input, and

Deployment Contract

version: int
repo_path: string
before_sha: string
after_sha: string
timestamp: string
runs: list

Runs List Entry

playbook: string
tags: string
priority: int
rule: string
file: string
reason: string
inventory: string
limit: string

GitOps Coordination

The GitOps Coordination section is a YAML-compliant block added to the release notes section for releases generated by the CI pipeline that allows multiple management pi nodes to coordinate and ensure jobs are executed once. It includes a statement of the jobs status, logging summary data including overall and per-command results, a claim expiration date that allows nodes to re-claim a job that may have failed during execution, and a nonce to avoid reliance on non-unique fields.

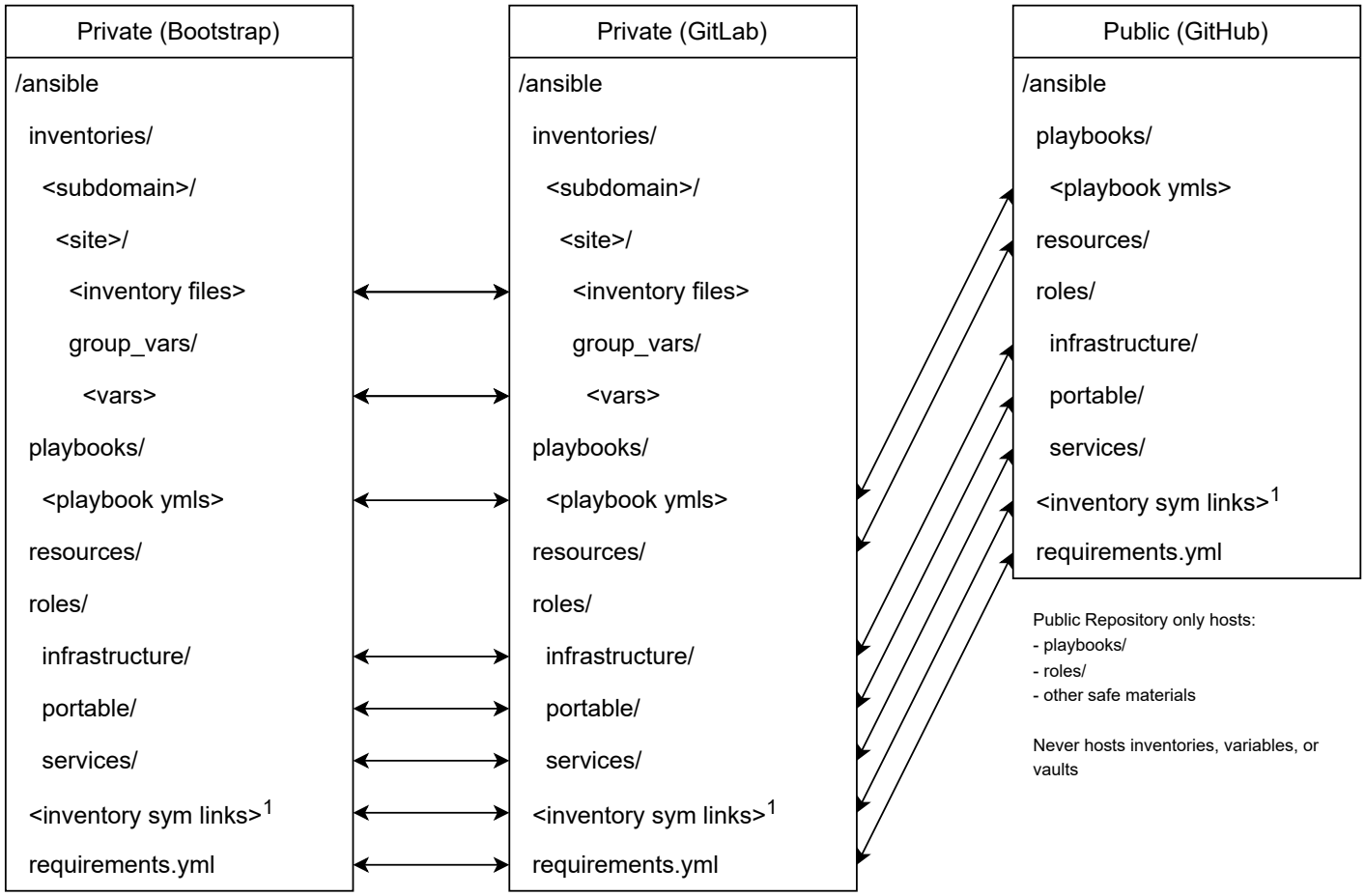
GitOps Section

[GITOPS_BEGIN]
gitops:
  status: string
  claimed_by: string
  expires_at: string
  claim_nonce: string
  result: string
  summary: list
[GITOPS_END]

Summary List Entry

command: string
rc: int
stderr: string
stdout: string

Infrastructure Repository Structure



* Private repos store vaulted vars, but no vault passwords, as an additional backup source.

[1] Inventory sym links map inventory files to the project root for easier access; i.e., ansible/inventories/production/core maps to ansible/production-core